

Autenticazione e Contesti (**oc login**, **oc config**)

Modulo: 2 — Navigazione e Strumenti

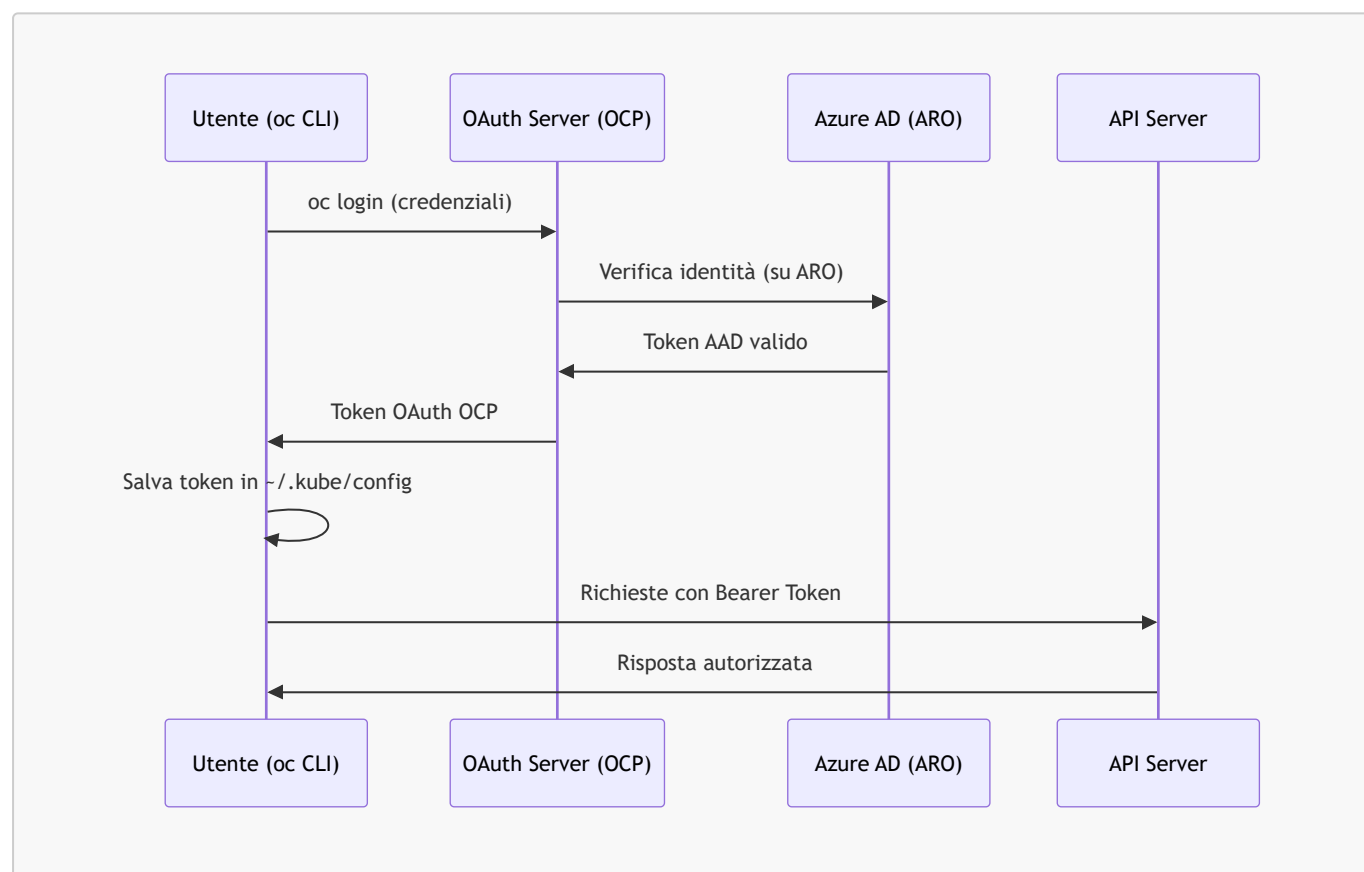
Versione: OpenShift 4.14+ su Azure ARO

Obiettivi di Apprendimento

- Autenticarsi a un cluster OCP/ARO con username/password, token e SSO Azure AD
- Comprendere la struttura del file kubeconfig e come viene gestito
- Gestire più cluster e contesti con **oc config** / **kubectl config**
- Applicare le best practice di sicurezza nella gestione dei token di accesso

1. Il Meccanismo di Autenticazione in OCP

OpenShift utilizza **OAuth 2.0** come meccanismo di autenticazione. Al login, l'API Server rilascia un **token** che viene salvato nel file kubeconfig (**~/.kube/config**) e usato per le richieste successive.



2. oc login — Autenticazione al Cluster

2.1 Login con Username e Password

```
# Sintassi base
oc login <API_URL> -u <username> -p <password>

# Esempio su ARO
oc login https://api.mycluster.region.aroapp.io:6443 \
  --username kubeadmin \
  --password <password-from-az-aro-list-credentials>

# Output atteso:
# Login successful.
# You have access to 68 projects, the list has been suppressed.
# You can list all projects with 'oc projects'
# Using project "default".
```

⚠ Attenzione: Non usare `--password` direttamente nel terminale su sistemi condivisi: il comando viene salvato nella history della shell. Preferire il prompt interattivo o il login tramite token.

```
# Login con prompt interattivo (più sicuro)
oc login https://api.mycluster.region.aroapp.io:6443
# Il terminale chiede username e password senza salvarle nella history
```

2.2 Login con Token OAuth

```
# Ottenere un token dalla console web:
# Console Web → (nome utente in alto a destra) → "Copy login command"
# → "Display Token" → copiare il comando completo

# Esempio
oc login --token=sha256~xxxxxxxxxxxxxxxxxx \
  --server=https://api.mycluster.region.aroapp.io:6443

# Output atteso:
# Logged into "https://api.mycluster.region.aroapp.io:6443"
# as "myuser@company.com" using the token provided.
```

2.3 Login con Azure Active Directory su ARO

Su ARO l'Identity Provider predefinito è **Azure Active Directory**. Il flusso OAuth avviene tramite browser:

```
# Avviare il login – si apre automaticamente il browser per l'autenticazione AAD
oc login https://api.mycluster.region.aroapp.io:6443

# Se il browser non si apre automaticamente, il comando fornisce un URL:
# Open <URL> in a browser to complete OAuth flow
# Inserire il codice mostrato nel terminale dopo l'autenticazione AAD
```

Credenziali ARO per accesso iniziale:

```
# Ottenere le credenziali kubeadmin tramite Azure CLI
az aro list-credentials \
  --name myAROCluster \
  --resource-group myResourceGroup
# Output:
# {
#   "kubeadminPassword": "xxxxx-xxxxx-xxxxx-xxxxx",
#   "kubeadminUsername": "kubeadmin"
# }
```

2.4 Verifica Utente Corrente

```
# Chi sono?
oc whoami
# Output: myuser@company.com

# Con quale token?
oc whoami --show-token
# Output: sha256~xxxxxxxxxxxxxxxxxxxx

# Informazioni complete sul cluster connesso
oc cluster-info
# Output:
# Kubernetes control plane is running at
https://api.mycluster.region.aroapp.io:6443
```

2.5 Logout

```
# Terminare la sessione (revoca il token sul server)
oc logout
# Output: Logged "myuser@company.com" out on
"https://api.mycluster.region.aroapp.io:6443"
```

3. Il File kubeconfig

Tutti i dati di connessione ai cluster vengono salvati nel file **kubeconfig**, di default in `~/.kube/config`.

Struttura del file kubeconfig

```
# ~/.kube/config – struttura semplificata
apiVersion: v1
kind: Config

# 1. Cluster: endpoint API e certificato CA
clusters:
- name: mycluster
  cluster:
    server: https://api.mycluster.region.aroapp.io:6443
    certificate-authority-data: <base64-encoded-CA>

# 2. Utenti: credenziali (token, certificato client)
users:
- name: myuser/mycluster
  user:
    token: sha256~xxxxxxxxxxxxxxxxxxxx

# 3. Contesti: combinazione cluster + utente + namespace
contexts:
- name: myuser/mycluster/default
  context:
    cluster: mycluster
    user: myuser/mycluster
    namespace: default

# 4. Contesto corrente
current-context: myuser/mycluster/default
```

Posizione e variabili d'ambiente

```
# Posizione default
~/.kube/config          # Linux/macOS
%USERPROFILE%\kube\config # Windows

# Override tramite variabile d'ambiente
export KUBECONFIG=/path/to/custom/config

# Unione di più file kubeconfig
export KUBECONFIG=~/.kube/config:~/.kube/config-cluster2

# Verificare quale kubeconfig è in uso
oc config view --minify
```

4. Gestione dei Contesti

4.1 Visualizzare i Contesti

```
# Elencare tutti i contesti disponibili
oc config get-contexts
# Output atteso:
# CURRENT  NAME                                     CLUSTER      AUTHINFO
NAMESPACE
# *        myuser/mycluster-aro/default      mycluster-aro
myuser/mycluster-aro  default
#          admin/cluster-prod/openshift-monitoring cluster-prod
admin/cluster-prod    openshift-monitoring

# Visualizzare il contesto corrente
oc config current-context
# Output: myuser/mycluster-aro/default
```

4.2 Cambiare Contesto

```
# Passare a un altro contesto (cluster diverso)
oc config use-context admin/cluster-prod/openshift-monitoring

# Cambiare solo il namespace del contesto corrente
oc project my-project
# oppure
oc config set-context --current --namespace=my-project
```

4.3 Aggiungere e Rimuovere Contesti

```
# Aggiungere manualmente un cluster al kubeconfig
oc config set-cluster cluster-dev \
  --server=https://api.cluster-dev.region.aroapp.io:6443 \
  --certificate-authority=/path/to/ca.crt

# Aggiungere un utente al kubeconfig
oc config set-credentials myuser-dev \
  --token=sha256~yyyyyyyyyyyyyyyyyy

# Creare un nuovo contesto
oc config set-context dev-context \
  --cluster=cluster-dev \
  --user=myuser-dev \
  --namespace=my-project

# Eliminare un contesto
oc config delete-context old-context

# Eliminare le credenziali di un utente
oc config delete-user old-user
```

4.4 Riepilogo Comandi `oc config`

```
oc config view                # mostra il kubeconfig completo
oc config view --minify       # mostra solo il contesto corrente
oc config get-contexts        # lista tutti i contesti
oc config current-context     # mostra il contesto attivo
oc config use-context <nome>  # cambia contesto attivo
oc config set-context --current --namespace=<ns> # cambia namespace
oc config get-clusters        # lista i cluster configurati
oc config get-users           # lista gli utenti configurati
```

5. Best Practice di Sicurezza

Gestione dei Token

Pratica	Descrizione
Non usare <code>kubeadmin</code> in produzione	Creare utenti nominali con RBAC appropriato; disabilitare <code>kubeadmin</code> dopo la configurazione AAD
Non condividere token	Ogni utente deve autenticarsi con le proprie credenziali
Scadenza dei token	I token OAuth OCP scadono dopo 24 ore per default; i token di ServiceAccount non scadono — monitorarli
Evitare token nella history	Usare <code>oc login</code> interattivo o variabili d'ambiente invece di <code>--password</code> e <code>--token</code> inline
Revocare i token non utilizzati	<code>oc logout</code> revoca il token sul server
Non committare kubeconfig	Aggiungere <code>~/.kube/config</code> al <code>.gitignore</code> dei progetti
Usare <code>--as</code> per impersonare	Testare permessi di altri utenti senza fare logout: <code>oc get pods --as=testuser</code>

Disabilitare `kubeadmin` su ARO dopo Configurazione AAD

```
# SOLO dopo aver verificato che l'accesso AAD funziona correttamente!  
# Eliminare il secret kubeadmin (operazione irreversibile)  
oc delete secret kubeadmin -n kube-system  
  
# Output atteso:  
# secret "kubeadmin" deleted
```

⚠ Attenzione: Eliminare `kubeadmin` è **irreversibile**. Eseguire questa operazione solo dopo aver verificato che almeno un utente AAD con ruolo `cluster-admin` può accedere correttamente.

6. Esercizi Pratici

Esercizio 1 — Login e Ispezione kubeconfig

Obiettivo: accedere al cluster ARO e comprendere il kubeconfig.

1. Ottenere l'URL API e le credenziali del cluster ARO:

```
az aro show --name myAROCluster --resource-group myRG \  
  --query "{api:apiserverProfile.url, console:consoleProfile.url}" -o table  
az aro list-credentials --name myAROCluster --resource-group myRG
```

2. Eseguire il login con `oc login`
3. Esaminare il kubeconfig generato:

```
cat ~/.kube/config  
oc config view --minify
```

4. Verificare l'utente corrente e il token:

```
oc whoami  
oc whoami --show-token
```

5. Esaminare a quali risorse ha accesso l'utente:

```
oc auth can-i --list
```

Risultato atteso: il kubeconfig contiene un blocco `cluster`, `user` e `context`. Il token è un hash `sha256~...`

Esercizio 2 — Gestione Multi-Cluster

Obiettivo: configurare e navigare tra due contesti distinti.

1. Dopo aver fatto login a un cluster, rinominare il contesto corrente:

```
oc config rename-context \  
  $(oc config current-context) \  
  aro-dev
```

2. Simulare un secondo cluster aggiungendo un contesto manuale:

```
oc config set-context aro-prod \  
  --cluster=$(oc config current-context | cut -d'/' -f2) \  
  --user=$(oc whoami) \  
  --namespace=production
```

3. Verificare i contesti disponibili e passare tra essi:

```
oc config get-contexts  
oc config use-context aro-prod  
oc project # quale progetto è attivo?  
oc config use-context aro-dev
```

Risultato atteso: `oc config get-contexts` mostra entrambi i contesti; il simbolo `*` indica quello attivo.

7. Riepilogo dei Punti Chiave

- `oc login` salva automaticamente le credenziali in `~/.kube/config` come **token OAuth**
- Su **ARO**, l'autenticazione avviene tramite **Azure Active Directory** — usare `oc login` con il browser per il flusso SSO
- Il file **kubeconfig** contiene: cluster (endpoint), utenti (token), contesti (combinazione cluster+utente+namespace)
- `oc config use-context` e `oc project` permettono di passare velocemente tra cluster e namespace
- **Best practice:** non usare `kubeadmin` in produzione, non condividere token, non committare kubeconfig nei repository

Riferimenti

- [Authenticating to OpenShift - Red Hat](#)
- [Configuring Azure AD on ARO - Microsoft](#)
- [Managing kubeconfig - Kubernetes](#)
- [oc login Reference](#)